ENTERPRISE ENGINEERING GUIDE

Windows 10 / 11 VDI Golden Template Implementation

Deploying a High-Density, Low-Latency Windows VDI Golden Image on Proxmox VE with VirtIO Drivers, Sysprep Generalization, and RDP / Guacamole Access

Proxmox Cluster/Host:	`prox000innspark (10.31.1.163)`
Guacamole Gateway:	`https://10.31.4.55/ (RDP Port 3389)`
Target OS:	`Windows 11 Enterprise (23H2/24H2) / Win 10 (22H2)`
Hypervisor Hardware:	`q35 | OVMF UEFI | vTPM 2.0 | VirtIO-SCSI`
Remote Transport:	`Remote Desktop (RDP / NLA) + SPICE QXL`
Driver Suite:	`Fedora VirtIO Drivers (vioscsi, NetKVM, Balloon)`
Automation Toolkit:	`optimize-and-seal-windows-vdi.ps1 + unattend.xml`
Storage Pool:	`local-lvm (LVM-Thin, Discard/TRIM=on, SSD=1)`
Execution Status:	**`Verified Production Standard`**

1. Overview and Architecture

Virtual Desktop Infrastructure (VDI) running Windows on Proxmox VE provides high compute consolidation, centralized data security, and instantaneous workstation cloning. However, consumer Windows operating systems running on shared hypervisors suffer from severe disk write amplification and CPU starvation unless properly paravirtualized and optimized.

This implementation establishes a hardened, optimized **Windows VDI Golden Image** utilizing the modern `q35` PCIe bus architecture, `OVMF (UEFI)` firmware, `vTPM 2.0` (swtpm), and paravirtualized `VirtIO` drivers. Users access their virtual desktops over high-performance **Remote Desktop Protocol (RDP)** directly or via an HTML5 browser session through **Apache Guacamole (10.31.4.55)**.

Layer	Component	Configuration Specification
Hypervisor	Proxmox VE 8.x	Host CPU emulation (`cpu=host`), IOThreads enabled, TRIM/Discard enabled.
Virtual Hardware	q35 & OVMF & vTPM	PCIe bus, 4MB EFI Disk with pre-enrolled keys, dedicated TPM 2.0 state.
Storage Controller	VirtIO SCSI Single	`virtio-scsi-single` with `iothread=1` and `discard=on,ssd=1`.
Network Adapter	VirtIO Network	`virtio` paravirtualized NIC bridged to `vmbr0` with optional VLAN.
Display & Audio	SPICE QXL & HDA	`qxl` display (64MB) + `ich9-intel-hda` audio for remote multimedia.
Remote Access	RDP & Guacamole	Port 3389 with NLA, managed by Apache Guacamole Gateway.
Image Sealing	Sysprep Generalization	Automated via `unattend.xml` with `PersistAllDeviceInstalls=true`.

2. Virtual Machine Hardware Profile

The build virtual machine is configured with production-grade virtualization parameters:

- **VM ID**: `9100` (Build VM ID, converted to immutable template upon sealing)
- **Processors**: 4 vCPUs (1 socket, 4 cores), Type: `host` (exposes AVX2/SSE4.2 instructions for browser video decode)
- **Memory**: 8192 MB (8 GB), Fixed RAM (Ballooning disabled `balloon=0` for consistent VDI latency)
- **System Disk**: 64 GB on `local-lvm` (VirtIO SCSI, SSD emulation, Discard/TRIM enabled)
- **Firmware & TPM**: OVMF (UEFI 4M) + vTPM 2.0 (Required for Windows 11 attestation & BitLocker)
- **Guest Agent**: QEMU Guest Agent enabled (`agent: enabled=1,fstrim_cloned_disks=1`)

> **Why Fixed RAM?** Dynamic memory ballooning in Windows desktop sessions can cause unpredictable micro-stuttering during active user sessions. Setting `balloon=0` ensures guaranteed response times across all VDI endpoints.

3. Step-by-Step Build VM Creation

The build VM can be instantiated instantly via the included Proxmox host bash automation script or executed manually in the PVE shell.

```
# Run automated build script on Proxmox VE Host Shell (as root)
/home/nishusaini/Desktop/Innspark/vdi-development/scripts/create-windows-vdi-build-vm.sh
```

Equivalent raw CLI commands executed by the script:

```
qm create 9100 \
    --name win11-vdi-gold-build \
    --machine q35 \
    --bios ovmf \
    --cpu host \
    --cores 4 \
    --sockets 1 \
    --memory 8192 \
    --balloon 0 \
    --scsihw virtio-scsi-single \
    --net0 virtio,bridge=vmbr0 \
    --agent enabled=1,fstrim_cloned_disks=1 \
    --vga qxl,memory=64 \
    --audio0 device=ich9-intel-hda,driver=spice \
    --ostype win11

# Add 4MB EFI Disk with Pre-Enrolled Keys
qm set 9100 --efidisk0 local-lvm:0,pre-enrolled-keys=1,efitype=4m

# Add Virtual TPM 2.0 State
qm set 9100 --tpmstate0 local-lvm:0,version=v2.0

# Create 64GB VirtIO SCSI Disk with TRIM and SSD Emulation
qm set 9100 --scsi0 local-lvm:64,discard=on,ssd=1,iothread=1

# Attach Windows Installation ISO (ide0) and VirtIO Drivers ISO (ide2)
qm set 9100 --ide0 local:iso/Win11_23H2_English_x64.iso,media=cdrom
qm set 9100 --ide2 local:iso/virtio-win.iso,media=cdrom

# Set Boot Sequence: Windows Installer ISO first, SCSI disk second
qm set 9100 --boot order=ide0;scsi0
```

4. Windows Installation & VirtIO Driver Loading

1. Start the VM in Proxmox: `qm start 9100`.
2. Open the Proxmox Console (SPICE or noVNC).
3. Press any key when prompted to boot from the CD-ROM.
4. In Windows Setup, select language/keyboard preferences and click **Install now**.
5. When reaching *"Where do you want to install Windows?"*, the drive list will be blank:
   - Click **Load driver** → click **Browse**.
   - Expand the **virtio-win** CD drive.
   - Browse to: `vioscsi` → `w11` (or `w10`) → `amd64`.
   - Select **Red Hat VirtIO SCSI controller** → click **Next**.

- The 64GB drive appears immediately.

6. Select **Drive 0 Unallocated Space** and click **Next** to install.

5. VirtIO Guest Tools & QEMU Agent Installation

Once booted into the Windows desktop:

1. Open **File Explorer** → navigate to **This PC** → double-click the **virtio-win** CD drive.
2. Run `virtio-win-guest-tools.exe`.
3. Proceed through the wizard with default selections. This installs:
   - **NetKVM** (VirtIO Ethernet adapter & driver)
   - **vioscsi** (VirtIO SCSI storage driver)
   - **vioser** (VirtIO Serial driver)
   - **BalloonService** (VirtIO memory management service)
   - **qemu-ga** (QEMU Guest Agent service)
   - **SPICE vdagent** (Clipboard sharing & dynamic resolution scaling)

> **Verification from Proxmox Host:** Verify guest agent health with `qm guest ping 9100`. When successful, the guest IP address is automatically visible in the Proxmox summary dashboard.

6. Remote Desktop (RDP) & Guacamole Integration

Enable Remote Desktop and configure high-performance audio redirection in PowerShell:

```
# Enable RDP and Network Level Authentication
Set-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Terminal Server" -Name "fDenyTSConnections" -
Set-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" -Name "U
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"

# Enable High Quality Audio & Microphone Redirection over RDP
Set-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" -Name "f
Set-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" -Name "f
```

Apache Guacamole Connection Parameters (10.31.4.55)

Parameter	Value	Description
Protocol	`rdp`	Standard Remote Desktop Protocol.
Hostname	`<CLONE_IP_ADDRESS>`	Assigned IP address of the cloned Windows VM.
Port	`3389`	Default RDP listening port.
Security mode	`NLA` (Network Level Authentication)	High-security TLS-authenticated session.
Ignore cert	`true`	Bypasses self-signed Windows RDP certificate warnings.
Enable audio	`true`	Streams high-fidelity audio directly to the web browser.
Color depth	`32-bit` or `24-bit`	Full 24-bit/32-bit TrueColor desktop experience.

7. Enterprise VDI Optimization & Debloating

Standard Windows includes consumer processes that create massive storage IOPS write amplification across virtual desktop pools. The optimization script applies the following core tunings:

- **Disable Hibernation** (`powercfg /h off`): Reclaims 8GB+ disk space (`hiberfil.sys`) and prevents Fast Startup corruption in linked clones.
- **Disable SysMain / Superfetch**: Eliminates write bursts to hypervisor SSD/SAN arrays.
- **Disable Windows Search Indexer (WSearch)**: Stops perpetual background indexing across clones.
- **Disable Scheduled Defragmentation**: Hypervisor TRIM/Discard manages SSD blocks automatically.
- **Disable Telemetry & Consumer Experiences**: Blocks automatic background installation of consumer games and advertising apps.
- **Visual Effects Tuning**: Disables window animations and drop shadows while retaining **Smooth Font Edges (ClearType)** for crisp readability.

8. Sysprep Generalization & Unattend.xml

To prevent SID collisions and ensure zero-touch clone initialization, the Golden Image is generalized with Microsoft Sysprep using a custom answer file.

> **CRITICAL SAFEGUARD: PersistAllDeviceInstalls**
> By default, `sysprep /generalize` uninstalls third-party drivers. Without `<PersistAllDeviceInstalls>true</PersistAllDeviceInstalls>`, Windows uninstalls the VirtIO SCSI controller, causing a fatal `INACCESSIBLE_BOOT_DEVICE` blue screen on clone boot. Our answer file ensures all VirtIO drivers remain permanently locked in the system.

Execute the all-in-one optimization and sealing suite inside Windows PowerShell as Administrator:

```
# Run automated guest optimization and Sysprep sealing
powershell.exe -ExecutionPolicy Bypass -File .\scripts\optimize-and-seal-windows-vdi.ps1
```

The script cleans AppX packages, purges event logs, resets the component store, stages `unattend.xml`, and initiates Sysprep with system shutdown:

```
sysprep.exe /generalize /oobe /shutdown /unattend:C:\Windows\System32\Sysprep\unattend.xml
```

9. Template Conversion & Instant Cloning

Once the build VM powers down following Sysprep:

```
# 1. Detach setup ISOs so clones do not carry installation media
qm set 9100 --delete ide0
qm set 9100 --delete ide2

# 2. Set boot order directly to primary SCSI disk
qm set 9100 --boot order=scsi0

# 3. Create a compressed safety backup
vzdump 9100 --mode stop --compress zstd --storage local

# 4. Convert VM 9100 into an immutable template
qm template 9100
```

Provisioning Linked Clones (Sub-10 Second Deployments)

```
# Create Linked Clones in seconds
qm clone 9100 9101 --name win11-vdi-01 --full 0
qm clone 9100 9102 --name win11-vdi-02 --full 0

# Start Clones
qm start 9101
qm start 9102
```

10. Multi-Clone Uniqueness Validation

To verify that Sysprep has properly generalized identities across clones, execute the verification script directly on the Proxmox host:

```
/home/nishusaini/Desktop/Innspark/vdi-development/scripts/verify-windows-clone-uniqueness.sh 9101 9102
```

Validation Metric	Expected State	Verification Command
QEMU Guest Agent	Online & Responding	`qm guest ping <VMID>`
ComputerName	Unique per clone	`$env:COMPUTERNAME` via guest exec
Machine SID	Strictly Unique	`[WindowsIdentity]::GetCurrent().User.AccountDomainSid`
Network MAC	Unique per clone	Proxmox `net0` interface config
IPv4 Address	Distinct DHCP Lease	`Get-NetIPAddress`
RDP Listener	Port 3389 Active	`Get-NetTCPConnection -LocalPort 3389`

11. Production Troubleshooting Matrix

Symptom / Error Code	Root Cause	Remediation Procedure
Drive not found in Windows Setup	Windows installer lacks VirtIO SCSI drivers.	Click *Load driver* → browse to `virtio-win\vioscsi\w11\amd64` → load driver.
Sysprep fails with 0x80073cf2	Windows Store updated an AppX package for current user.	Run `optimize-and-seal-windows-vdi.ps1` to remove per-user packages before sealing.
BSOD 0x7B: INACCESSIBLE_BOOT_DEVICE	Sysprep stripped VirtIO SCSI controller driver.	Ensure `unattend.xml` has `<PersistAllDeviceInstalls>true</PersistAllDeviceInstalls>`.
QEMU Agent shows offline in PVE UI	VirtIO Serial driver or service stopped.	Run `virtio-win-guest-tools.exe` inside guest and set `qemu-ga` to Automatic.
Black screen on RDP connection	Display virtualization conflict.	Install `qxldod` driver from VirtIO ISO or enable WDDM graphics via Group Policy.

Engineering Delivery Summary

Complete Windows VDI automation suite operational: Proxmox Provisioner, Sysprep Answer File, PowerShell Optimization Suite, and Uniqueness Verifier.